GOVERNMENT OF INDIA • COPYRIGHT OFFICE (FORM XIV)

STATEMENT OF PARTICULARS — COMPUTER SOFTWARE DEPOSIT

ShieldIntern: Real-Time Recruitment Fraud Detection and Offer Forensics Software

Complete Technical Source Code & Architectural Blueprint Repository

Title of Work:	ShieldIntern: Real-Time Recruitment Fraud Detection and Offer Forensics Software
Class of Work:	Computer Software (Literary Work under Section 2(o) of The Copyright Act, 1957)
Authors & Owners:	1. Soham Jadhav (Lead Architect & Developer) 2. Amey Bauchkar (Systems & Ingestion Engineer)
Academic Department:	Department of Computer Engineering & Information Technology, University of Mumbai, Maharashtra, India
Primary Languages:	TypeScript, Node.js (v20+), Next.js 14, React 18, Tailwind CSS
First Publication / Creation:	October 2026 (Unpublished Proprietary Source Code / All Rights Reserved)
Software Architecture:	Event-Driven Ingestion, Tiered LPU Transformer Inference, DNS Heuristic Matrix
Filing Purpose:	Mandatory Literary Work Document (PDF < 10 MB) for Copyright Registration

Architecture Preamble & Implementation Notes

Developer Note: ShieldIntern was conceived and engineered by Soham Jadhav and Amey Bauchkar to address the alarming surge in fraudulent internship and campus placement solicitations across Indian technical universities. Modern fraudsters purchase disposable lookalike domains and request UPI security deposits (for laptop dispatch, onboarding clearances, or mock training modules) without weaponizing malicious binary attachments.

To provide sub-second forensic triage without compromising student confidentiality, the codebase implements an asynchronous dual-engine pipeline: (1) An event-driven MIME listener staging incoming communications in an isolated in-memory quarantine queue, (2) A Human-in-the-Loop (HITL) consent barrier ensuring no student email is transmitted without explicit approval, (3) A tiered multi-provider LLM inference engine routing requests to Groq LPUs with transparent failover to Google Gemini, and (4) A deterministic composite scoring function fusing semantic scores with WHOIS domain age heuristics and regex financial extortion filters.

Source Code Manifest & File Index

#	File Path & Module	Lines	Functional Role
1	src/lib/types.ts	110	Data contracts, threat taxonomies, scoring models
2	src/lib/domainCheck.ts	185	DNS age heuristics, typosquatting & TLD vulnerability
3	src/lib/incomingStore.ts	95	Ephemeral in-memory quarantine queue & HITL staging
4	src/app/api/analyze/route.ts	280	Core forensic triage, tiered LPU inference, composite scoring

5	src/app/api/n8n/incoming/route.ts	120	Webhook ingestion endpoint, SHA-256 deduplication
6	src/components/views/XRayTextView.tsx	165	Client-side interactive X-Ray predatory text highlighter
7	src/store/offerStore.ts	130	State management, reactive forensic metrics & complaints

Module 1: Core Type Definitions & Threat Taxonomy

File Path:	src/lib/types.ts
Module Role:	Establishes strict TypeScript interfaces for forensic analysis responses, heuristic risk flags, DNS reputation structures, and HITL quarantined email payloads.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	56 lines of code | TypeScript / Next.js

```
0001 | export interface HighlightedPhrase {
0002 |   text: string;
0003 |   severity: "safe" | "suspicious" | "danger";
0004 | }
0005 |
0006 | export interface FraudIndicator {
0007 |   icon: string;
0008 |   title: string;
0009 |   severity: "low" | "medium" | "high";
0010 |   description: string;
0011 | }
0012 |
0013 | export interface AnalysisResult {
0014 |   riskScore: number;
0015 |   riskLevel: "safe" | "suspicious" | "high-risk";
0016 |   summary: string;
0017 |   highlights: HighlightedPhrase[];
0018 |   indicators: FraudIndicator[];
0019 |   verificationSteps: string[];
0020 |   extractedEntities?: {
0021 |     companyName: string;
0022 |     contactEmail: string;
0023 |     contactPhone: string;
0024 |     amountRequested: string;
0025 |     dateReceived: string;
0026 |   };
0027 |   complaintDraft?: string;
0028 | }
0029 |
0030 | export interface MockOffer {
0031 |   id: string;
0032 |   title: string;
0033 |   preview: string;
0034 |   text: string;
0035 |   tag: "scam" | "suspicious" | "safe";
0036 |   extractedEntities?: {
0037 |     companyName: string;
0038 |     contactEmail: string;
0039 |     contactPhone: string;
0040 |     amountRequested: string;
0041 |     dateReceived: string;
0042 |   };
0043 |   complaintDraft?: string;
0044 | }
0045 |
0046 | export interface IncomingOffer {
0047 |   id: string;
0048 |   messageId?: string;
0049 |   sender: string;
0050 |   subject: string;
0051 |   snippet: string;
0052 |   body: string;
0053 |   receivedAt: string;
0054 |   timestamp: number;
0055 | }
0056 |
```

Module 2: DNS Heuristics & Domain Age Intelligence

File Path:	src/lib/domainCheck.ts
Module Role:	Implements RDAP/WHOIS lookup queries, newly registered domain (NRD) age thresholds (< 30 days penalty), and free-mail / typo-squatting detection.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	489 lines of code | TypeScript / Next.js

```
0001 | /**
0002 |  * Domain Legitimacy Checker
0003 |  *
0004 |  * Standalone module that verifies whether a domain used in a job/internship
0005 |  * offer is legitimate or potentially fraudulent. Runs 6 independent checks:
0006 |  *   1. Free email provider detection
0007 |  *   2. Sender domain vs claimed domain match
0008 |  *   3. RDAP domain age lookup (free, no API key)
0009 |  *   4. MX record verification (Node dns module)
0010 |  *   5. Typosquat detection (Levenshtein distance vs known companies)
0011 |  *   6. Verdict computation with plain-English reasons
0012 |  *
0013 |  * This file is self-contained — it does NOT import from or modify any
0014 |  * existing project files (types.ts, offerStore.ts, etc.).
0015 |  *
0016 |  * INTEGRATION NOTES:
0017 |  * -  To display results in the UI, a teammate should create a component
0018 |  *    that calls the /api/domain-check POST endpoint and renders the
0019 |  *    DomainCheckResult. This can be added below the existing
0020 |  *    ResultsDashboard in src/components/views/ResultsDashboard.tsx,
0021 |  *    gated on result.riskLevel === "high-risk".
0022 |  * -  The AnalysisResult interface in src/lib/types.ts could optionally
0023 |  *    gain a `domainCheck?: DomainCheckResult` field, but do NOT add it
0024 |  *    without team sign-off.
0025 |  */
0026 |
0027 | import dns from "dns";
0028 |
0029 | // ═══════════════════════════════════════════════════════════════════════
0030 | // Public Interface
0031 | // ═══════════════════════════════════════════════════════════════════════
0032 |
0033 | /**
0034 |  * Result returned by `checkDomainLegitimacy`.
0035 |  *
0036 |  * @field domain             - The domain that was checked (e.g. "globaltech-solutions.xyz").
0037 |  * @field claimedCompany     - The company name the sender claims to represent.
0038 |  * @field senderDomainMatch  - true if the sender email's domain matches `domain` exactly.
0039 |  * @field isFreeEmailProvider - true if sender uses gmail, yahoo, outlook, etc.
0040 |  * @field domainAgeDays      - Days since domain was registered. null if lookup failed.
0041 |  * @field domainCreatedDate  - ISO date string of domain creation. null if lookup failed.
0042 |  * @field isTyposquat        - true if `domain` is suspiciously similar to a known company domain.
0043 |  * @field typosquatTarget    - The real domain being impersonated, or null.
0044 |  * @field hasMxRecords       - true if the domain has valid MX (mail server) records.
0045 |  * @field verdict            - Final assessment: 'legit' | 'suspicious' | 'likely-fake'.
0046 |  * @field reasons            - Array of plain-English explanations for every flagged issue.
0047 |  */
0048 | export interface DomainCheckResult {
0049 |   domain: string;
0050 |   claimedCompany: string;
0051 |   senderDomainMatch: boolean;
0052 |   isFreeEmailProvider: boolean;
0053 |   domainAgeDays: number | null;
0054 |   domainCreatedDate: string | null;
0055 |   isTyposquat: boolean;
0056 |   typosquatTarget: string | null;
0057 |   hasMxRecords: boolean;
0058 |   verdict: "legit" | "suspicious" | "likely-fake";
0059 |   reasons: string[];
0060 | }
0061 |
0062 | // ═══════════════════════════════════════════════════════════════════════
0063 | // Constants
0064 | // ═══════════════════════════════════════════════════════════════════════
0065 |
0066 | /** Free email providers commonly used in India */
0067 | const FREE_EMAIL_PROVIDERS = new Set([
0068 |   "gmail.com",
0069 |   "yahoo.com",
0070 |   "outlook.com",
0071 |   "hotmail.com",
0072 |   "rediffmail.com",
0073 |   "protonmail.com",
0074 |   "yahoo.co.in",
0075 |   "yahoo.in",
```

```
0076 |   "live.com",
0077 |   "aol.com",
0078 | ]);
0079 |
0080 | /**
0081 |  * Well-known companies relevant to Indian campus placements,
0082 |  * mapped to their real primary domain.
0083 |  */
0084 | const KNOWN_COMPANY_DOMAINS: { company: string; domain: string }[] = [
0085 |   { company: "Infosys", domain: "infosys.com" },
0086 |   { company: "TCS", domain: "tcs.com" },
0087 |   { company: "Wipro", domain: "wipro.com" },
0088 |   { company: "Accenture", domain: "accenture.com" },
0089 |   { company: "Cognizant", domain: "cognizant.com" },
0090 |   { company: "Capgemini", domain: "capgemini.com" },
0091 |   { company: "HCL", domain: "hcltech.com" },
0092 |   { company: "Tech Mahindra", domain: "techmahindra.com" },
0093 |   { company: "IBM", domain: "ibm.com" },
0094 |   { company: "Google", domain: "google.com" },
0095 |   { company: "Microsoft", domain: "microsoft.com" },
0096 |   { company: "Amazon", domain: "amazon.com" },
0097 |   { company: "Deloitte", domain: "deloitte.com" },
0098 |   { company: "EY", domain: "ey.com" },
0099 |   { company: "KPMG", domain: "kpmg.com" },
0100 | ];
0101 |
0102 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0103 | // Levenshtein Distance (local implementation — no npm dependency)
0104 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0105 |
0106 | /**
0107 |  * Compute the Levenshtein edit-distance between two strings.
0108 |  * Uses the classic dynamic-programming approach.
0109 |  */
0110 | function levenshtein(a: string, b: string): number {
0111 |   const la = a.length;
0112 |   const lb = b.length;
0113 |   const dp: number[][] = Array.from({ length: la + 1 }, () =>
0114 |     Array(lb + 1).fill(0)
0115 |   );
0116 |
0117 |   for (let i = 0; i <= la; i++) dp[i][0] = i;
0118 |   for (let j = 0; j <= lb; j++) dp[0][j] = j;
0119 |
0120 |   for (let i = 1; i <= la; i++) {
0121 |     for (let j = 1; j <= lb; j++) {
0122 |       const cost = a[i - 1] === b[j - 1] ? 0 : 1;
0123 |       dp[i][j] = Math.min(
0124 |         dp[i - 1][j] + 1,      // deletion
0125 |         dp[i][j - 1] + 1,      // insertion
0126 |         dp[i - 1][j - 1] + cost // substitution
0127 |       );
0128 |     }
0129 |   }
0130 |
0131 |   return dp[la][lb];
0132 | }
0133 |
0134 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0135 | // Individual Checks
0136 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0137 |
0138 | /** Extract the domain portion from an email address. */
0139 | function extractEmailDomain(email: string): string {
0140 |   const parts = email.split("@");
0141 |   return (parts[1] || "").toLowerCase().trim();
0142 | }
0143 |
0144 | /** Check 1 & 2: Free email provider + sender domain match */
0145 | function checkEmailBasics(
0146 |   domain: string,
0147 |   senderEmail: string
0148 | ): { isFreeEmailProvider: boolean; senderDomainMatch: boolean; senderDomain: string } {
0149 |   const senderDomain = extractEmailDomain(senderEmail);
0150 |   return {
```

```
0151 |     isFreeEmailProvider: FREE_EMAIL_PROVIDERS.has(senderDomain),
0152 |     senderDomainMatch: senderDomain === domain.toLowerCase().trim(),
0153 |     senderDomain,
0154 |   };
0155 | }
0156 |
0157 | /**
0158 |  * Check 3: RDAP domain age lookup.
0159 |  * Uses the free RDAP protocol — no API key required.
0160 |  * Returns { domainAgeDays, domainCreatedDate } or nulls on failure.
0161 |  */
0162 | async function checkDomainAge(
0163 |   domain: string
0164 | ): Promise<{ domainAgeDays: number | null; domainCreatedDate: string | null; error?: string }> {
0165 |   try {
0166 |     const response = await fetch(`https://rdap.org/domain/${domain}`, {
0167 |       signal: AbortSignal.timeout(8000), // 8s timeout
0168 |     });
0169 |
0170 |     if (!response.ok) {
0171 |       return {
0172 |         domainAgeDays: null,
0173 |         domainCreatedDate: null,
0174 |         error: `RDAP returned status ${response.status}`,
0175 |       };
0176 |     }
0177 |
0178 |     const data = await response.json();
0179 |     const events = data.events as
0180 |       | { eventAction: string; eventDate: string }[]
0181 |       | undefined;
0182 |
0183 |     if (!Array.isArray(events)) {
0184 |       return {
0185 |         domainAgeDays: null,
0186 |         domainCreatedDate: null,
0187 |         error: "No events array in RDAP response",
0188 |       };
0189 |     }
0190 |
0191 |     const registrationEvent = events.find(
0192 |       (e) => e.eventAction === "registration"
0193 |     );
0194 |
0195 |     if (!registrationEvent?.eventDate) {
0196 |       return {
0197 |         domainAgeDays: null,
0198 |         domainCreatedDate: null,
0199 |         error: "No registration event found in RDAP data",
0200 |       };
0201 |     }
0202 |
0203 |     const createdDate = new Date(registrationEvent.eventDate);
0204 |     const now = new Date();
0205 |     const ageDays = Math.floor(
0206 |       (now.getTime() - createdDate.getTime()) / (1000 * 60 * 60 * 24)
0207 |     );
0208 |
0209 |     return {
0210 |       domainAgeDays: ageDays,
0211 |       domainCreatedDate: createdDate.toISOString().split("T")[0],
0212 |     };
0213 |   } catch (err) {
0214 |     return {
0215 |       domainAgeDays: null,
0216 |       domainCreatedDate: null,
0217 |       error: err instanceof Error ? err.message : String(err),
0218 |     };
0219 |   }
0220 | }
0221 |
0222 | /**
0223 |  * Check 4: MX record verification using Node's built-in dns module.
0224 |  * Returns true if at least one MX record exists for the domain.
0225 |  */
```

```
0226 | async function checkMxRecords(domain: string): Promise<boolean> {
0227 |   try {
0228 |     const records = await dns.promises.resolveMx(domain);
0229 |     return Array.isArray(records) && records.length > 0;
0230 |   } catch {
0231 |     // ENOTFOUND, ENODATA, etc. — domain has no MX records
0232 |     return false;
0233 |   }
0234 | }
0235 |
0236 | /**
0237 |  * Check 5: Typosquat detection.
0238 |  * Compares the domain against well-known company domains using
0239 |  * Levenshtein distance. Flags if distance is 1–3 but NOT exact match.
0240 |  */
0241 | function checkTyposquat(
0242 |   domain: string
0243 | ): { isTyposquat: boolean; typosquatTarget: string | null; matchedCompany: string | null } {
0244 |   const domainLower = domain.toLowerCase().trim();
0245 |
0246 |   for (const entry of KNOWN_COMPANY_DOMAINS) {
0247 |     const realDomain = entry.domain.toLowerCase();
0248 |
0249 |     // Skip exact matches — exact match means it IS the real domain
0250 |     if (domainLower === realDomain) {
0251 |       return { isTyposquat: false, typosquatTarget: null, matchedCompany: null };
0252 |     }
0253 |
0254 |     const distance = levenshtein(domainLower, realDomain);
0255 |
0256 |     if (distance >= 1 && distance <= 3) {
0257 |       return {
0258 |         isTyposquat: true,
0259 |         typosquatTarget: entry.domain,
0260 |         matchedCompany: entry.company,
0261 |       };
0262 |     }
0263 |   }
0264 |
0265 |   return { isTyposquat: false, typosquatTarget: null, matchedCompany: null };
0266 | }
0267 |
0268 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0269 | // Verdict & Reasons
0270 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0271 |
0272 | /**
0273 |  * Check 6: Compute final verdict and build reasons array.
0274 |  */
0275 | function computeVerdict(params: {
0276 |   isFreeEmailProvider: boolean;
0277 |   senderDomainMatch: boolean;
0278 |   senderDomain: string;
0279 |   domainAgeDays: number | null;
0280 |   domainCreatedDate: string | null;
0281 |   domainAgeError?: string;
0282 |   isTyposquat: boolean;
0283 |   typosquatTarget: string | null;
0284 |   matchedCompany: string | null;
0285 |   hasMxRecords: boolean;
0286 |   domain: string;
0287 |   claimedCompany: string;
0288 | }): { verdict: "legit" | "suspicious" | "likely-fake"; reasons: string[] } {
0289 |   const reasons: string[] = [];
0290 |   let isLikelyFake = false;
0291 |   let isSuspicious = false;
0292 |
0293 |   // ■■ Typosquat ■■
0294 |   if (params.isTyposquat) {
0295 |     isLikelyFake = true;
0296 |     reasons.push(
0297 |       `The domain "${params.domain}" looks very similar to the real "${params.typosquatTarget}" (used by ${params.matche
0298 |     );
0299 |   }
0300 |
```

```
0301 |   // ■■ Free email + domain mismatch ■■
0302 |   if (params.isFreeEmailProvider && !params.senderDomainMatch) {
0303 |     isLikelyFake = true;
0304 |     reasons.push(
0305 |       `The sender is using a free email provider (${params.senderDomain}) instead of a company email address. Legitimate
0306 |     );
0307 |   } else if (params.isFreeEmailProvider && params.senderDomainMatch) {
0308 |     // Edge case: claimed domain IS a free provider (very odd for a company)
0309 |     isSuspicious = true;
0310 |     reasons.push(
0311 |       `The sender email uses a free email provider (${params.senderDomain}). Real companies send official communications
0312 |     );
0313 |   }
0314 |
0315 |   // ■■ Domain age ■■
0316 |   if (params.domainAgeDays !== null) {
0317 |     if (params.domainAgeDays < 30) {
0318 |       isLikelyFake = true;
0319 |       reasons.push(
0320 |         `This domain was registered only ${params.domainAgeDays} day${params.domainAgeDays === 1 ? "" : "s"} ago (on ${p
0321 |       );
0322 |     } else if (params.domainAgeDays < 180) {
0323 |       isSuspicious = true;
0324 |       reasons.push(
0325 |         `This domain was registered ${params.domainAgeDays} days ago (on ${params.domainCreatedDate}), which is relative
0326 |       );
0327 |     } else {
0328 |       reasons.push(
0329 |         `The domain has been registered for ${params.domainAgeDays} days (since ${params.domainCreatedDate}), which is a
0330 |       );
0331 |     }
0332 |   } else {
0333 |     isSuspicious = true;
0334 |     reasons.push(
0335 |       params.domainAgeError
0336 |         ? `Domain could not be verified via RDAP — may not exist or lookup failed (${params.domainAgeError}). Be cautiou
0337 |         : `Domain could not be verified via RDAP — may not exist or lookup failed. Be cautious when a domain's registrat
0338 |     );
0339 |   }
0340 |
0341 |   // ■■ MX records ■■
0342 |   if (!params.hasMxRecords) {
0343 |     isSuspicious = true;
0344 |     reasons.push(
0345 |       `No mail servers (MX records) are configured for "${params.domain}". This means the domain cannot actually send or
0346 |     );
0347 |   }
0348 |
0349 |   // ■■ Sender domain mismatch (non-free) ■■
0350 |   if (!params.senderDomainMatch && !params.isFreeEmailProvider) {
0351 |     isSuspicious = true;
0352 |     reasons.push(
0353 |       `The sender's email domain (${params.senderDomain}) does not match the claimed company domain (${params.domain}).
0354 |     );
0355 |   }
0356 |
0357 |   // ■■ Determine verdict ■■
0358 |   let verdict: "legit" | "suspicious" | "likely-fake";
0359 |   if (isLikelyFake) {
0360 |     verdict = "likely-fake";
0361 |   } else if (isSuspicious) {
0362 |     verdict = "suspicious";
0363 |   } else {
0364 |     verdict = "legit";
0365 |   }
0366 |
0367 |   // If legit and no reasons beyond the positive age one, add a reassuring note
0368 |   if (verdict === "legit" && reasons.length <= 1) {
0369 |     reasons.push(
0370 |       `The domain appears to be legitimately configured with proper mail servers. The sender's email matches the claimed
0371 |     );
0372 |   }
0373 |
0374 |   return { verdict, reasons };
0375 | }
```

```
0376 |
0377 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0378 | // Main Exported Function
0379 | // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
0380 |
0381 | /**
0382 |  * Checks the legitimacy of a domain used in a job/internship offer.
0383 |  *
0384 |  * Runs 6 independent checks (free email, domain match, RDAP age,
0385 |  * MX records, typosquat detection) and returns a comprehensive result
0386 |  * with a final verdict and plain-English reasons.
0387 |  *
0388 |  * **Never throws** — all errors are caught internally and result in
0389 |  * conservative "suspicious" verdicts with explanatory reasons.
0390 |  *
0391 |  * @param params.domain         - The domain to verify (e.g. "globaltech-solutions.xyz")
0392 |  * @param params.claimedCompany - The company name from the offer (e.g. "GlobalTech Solutions")
0393 |  * @param params.senderEmail    - The full sender email address (e.g. "hr@globaltech-solutions.xyz")
0394 |  *
0395 |  * @returns A DomainCheckResult with all check findings and final verdict.
0396 |  */
0397 | export async function checkDomainLegitimacy(params: {
0398 |   domain: string;
0399 |   claimedCompany: string;
0400 |   senderEmail: string;
0401 | }): Promise<DomainCheckResult> {
0402 |   const { domain, claimedCompany, senderEmail } = params;
0403 |
0404 |   try {
0405 |     // ■■ Known company whitelist — skip unreliable checks for verified domains ■■
0406 |     const knownDomain = KNOWN_COMPANY_DOMAINS.find(
0407 |       (entry) => entry.domain.toLowerCase() === domain.toLowerCase().trim()
0408 |     );
0409 |     if (knownDomain) {
0410 |       const emailBasics = checkEmailBasics(domain, senderEmail);
0411 |       return {
0412 |         domain,
0413 |         claimedCompany,
0414 |         senderDomainMatch: emailBasics.senderDomainMatch,
0415 |         isFreeEmailProvider: emailBasics.isFreeEmailProvider,
0416 |         domainAgeDays: null,
0417 |         domainCreatedDate: null,
0418 |         isTyposquat: false,
0419 |         typosquatTarget: null,
0420 |         hasMxRecords: true,
0421 |         verdict: "legit",
0422 |         reasons: [
0423 |           `"${domain}" is the verified official domain of ${knownDomain.company}. This is a well-known, established comp
0424 |           ...(emailBasics.senderDomainMatch
0425 |             ? [`The sender's email matches the official ${knownDomain.company} domain — this is a good sign.`]
0426 |             : [`Note: The sender's email (${emailBasics.senderDomain}) does not match ${domain}. Verify that the email i
0427 |         ],
0428 |       };
0429 |     }
0430 |
0431 |     // Run all checks — independent, so we can parallelize the async ones
0432 |     const emailBasics = checkEmailBasics(domain, senderEmail);
0433 |     const typosquatResult = checkTyposquat(domain);
0434 |
0435 |     // RDAP and DNS are I/O — run in parallel
0436 |     const [ageResult, hasMxRecords] = await Promise.all([
0437 |       checkDomainAge(domain),
0438 |       checkMxRecords(domain),
0439 |     ]);
0440 |
0441 |     // Compute verdict
0442 |     const { verdict, reasons } = computeVerdict({
0443 |       isFreeEmailProvider: emailBasics.isFreeEmailProvider,
0444 |       senderDomainMatch: emailBasics.senderDomainMatch,
0445 |       senderDomain: emailBasics.senderDomain,
0446 |       domainAgeDays: ageResult.domainAgeDays,
0447 |       domainCreatedDate: ageResult.domainCreatedDate,
0448 |       domainAgeError: ageResult.error,
0449 |       isTyposquat: typosquatResult.isTyposquat,
0450 |       typosquatTarget: typosquatResult.typosquatTarget,
```

```
0451 |       matchedCompany: typosquatResult.matchedCompany,
0452 |       hasMxRecords,
0453 |       domain,
0454 |       claimedCompany,
0455 |     });
0456 |
0457 |     return {
0458 |       domain,
0459 |       claimedCompany,
0460 |       senderDomainMatch: emailBasics.senderDomainMatch,
0461 |       isFreeEmailProvider: emailBasics.isFreeEmailProvider,
0462 |       domainAgeDays: ageResult.domainAgeDays,
0463 |       domainCreatedDate: ageResult.domainCreatedDate,
0464 |       isTyposquat: typosquatResult.isTyposquat,
0465 |       typosquatTarget: typosquatResult.typosquatTarget,
0466 |       hasMxRecords,
0467 |       verdict,
0468 |       reasons,
0469 |     };
0470 |   } catch (err) {
0471 |     // Catastrophic fallback — should never reach here, but just in case
0472 |     console.error("checkDomainLegitimacy fatal error:", err);
0473 |     return {
0474 |       domain,
0475 |       claimedCompany,
0476 |       senderDomainMatch: false,
0477 |       isFreeEmailProvider: false,
0478 |       domainAgeDays: null,
0479 |       domainCreatedDate: null,
0480 |       isTyposquat: false,
0481 |       typosquatTarget: null,
0482 |       hasMxRecords: false,
0483 |       verdict: "suspicious",
0484 |       reasons: [
0485 |         "Domain verification could not be completed due to an internal error. Treat this offer with caution and verify t
0486 |       ],
0487 |     };
0488 |   }
0489 | }
```

Module 3: Ephemeral Quarantine Store & HITL Staging

File Path:	src/lib/incomingStore.ts
Module Role:	Privacy-preserving in-memory queue that holds intercepted candidate emails in an isolated state until the student grants explicit manual consent.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	190 lines of code | TypeScript / Next.js

```
0001 | import { IncomingOffer } from "./types";
0002 | import { Redis } from "@upstash/redis";
0003 |
0004 | // Check for Upstash Redis or Vercel KV environment variables
0005 | function getRedisClient(): Redis | null {
0006 |   const url =
0007 |     process.env.UPSTASH_REDIS_REST_URL ||
0008 |     process.env.KV_REST_API_URL ||
0009 |     process.env.STORAGE_REST_API_URL ||
0010 |     process.env.STORAGE_URL;
0011 |   const token =
0012 |     process.env.UPSTASH_REDIS_REST_TOKEN ||
0013 |     process.env.KV_REST_API_TOKEN ||
0014 |     process.env.STORAGE_REST_API_TOKEN ||
0015 |     process.env.STORAGE_TOKEN;
0016 |
0017 |   if (url && token) {
0018 |     try {
0019 |       return new Redis({ url, token });
0020 |     } catch (e) {
0021 |       console.warn("Failed to initialize Upstash Redis:", e);
0022 |     }
0023 |   }
0024 |
0025 |   try {
0026 |     return Redis.fromEnv();
0027 |   } catch {
0028 |     return null;
0029 |   }
0030 | }
0031 |
0032 | // In-memory fallback queue for local development
0033 | const memoryPendingOffers: Map<string, IncomingOffer> = new Map();
0034 | const memoryProcessedIds: Set<string> = new Set();
0035 |
0036 | const REDIS_PENDING_KEY = "shieldintern:pending_offers";
0037 | const REDIS_PROCESSED_KEY = "shieldintern:processed_ids";
0038 |
0039 | export const incomingStore = {
0040 |   async add(offer: Omit<IncomingOffer, "id" | "timestamp">): Promise<IncomingOffer | null> {
0041 |     const messageId = offer.messageId;
0042 |     const redis = getRedisClient();
0043 |
0044 |     if (redis) {
0045 |       try {
0046 |         // 1. Check if already processed
0047 |         if (messageId) {
0048 |           const isProcessed = await redis.sismember(REDIS_PROCESSED_KEY, messageId);
0049 |           if (isProcessed) return null;
0050 |         }
0051 |
0052 |         // 2. Check if already in pending queue
0053 |         if (messageId) {
0054 |           const allPending = (await redis.hvals<IncomingOffer>(REDIS_PENDING_KEY)) || [];
0055 |           if (allPending.some((o) => o.messageId === messageId)) {
0056 |             return null;
0057 |           }
0058 |         }
0059 |
0060 |         const id = `offer_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`;
0061 |         const fullOffer: IncomingOffer = {
0062 |           ...offer,
0063 |           id,
0064 |           timestamp: Date.now(),
0065 |         };
0066 |
0067 |         await redis.hset(REDIS_PENDING_KEY, { [id]: fullOffer });
0068 |         return fullOffer;
0069 |       } catch (err) {
0070 |         console.warn("Upstash Redis add error, falling back to in-memory:", err);
0071 |       }
0072 |     }
0073 |
0074 |     // In-memory fallback
0075 |     if (messageId && memoryProcessedIds.has(messageId)) {
```

```
0076 |       return null;
0077 |     }
0078 |     if (messageId) {
0079 |       for (const existing of memoryPendingOffers.values()) {
0080 |         if (existing.messageId === messageId) return null;
0081 |       }
0082 |     }
0083 |
0084 |     const id = `offer_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`;
0085 |     const fullOffer: IncomingOffer = {
0086 |       ...offer,
0087 |       id,
0088 |       timestamp: Date.now(),
0089 |     };
0090 |     memoryPendingOffers.set(id, fullOffer);
0091 |
0092 |     if (memoryPendingOffers.size > 25) {
0093 |       const oldestKey = memoryPendingOffers.keys().next().value;
0094 |       if (oldestKey) memoryPendingOffers.delete(oldestKey);
0095 |     }
0096 |
0097 |     return fullOffer;
0098 |   },
0099 |
0100 |   async getAll(): Promise<IncomingOffer[]> {
0101 |     const redis = getRedisClient();
0102 |     if (redis) {
0103 |       try {
0104 |         const offers = (await redis.hvals<IncomingOffer>(REDIS_PENDING_KEY)) || [];
0105 |         return offers.sort((a, b) => b.timestamp - a.timestamp);
0106 |       } catch (err) {
0107 |         console.warn("Upstash Redis getAll error, falling back to in-memory:", err);
0108 |       }
0109 |     }
0110 |
0111 |     return Array.from(memoryPendingOffers.values()).sort((a, b) => b.timestamp - a.timestamp);
0112 |   },
0113 |
0114 |   async get(id: string): Promise<IncomingOffer | undefined> {
0115 |     const redis = getRedisClient();
0116 |     if (redis) {
0117 |       try {
0118 |         const offer = await redis.hget<IncomingOffer>(REDIS_PENDING_KEY, id);
0119 |         return offer || undefined;
0120 |       } catch (err) {
0121 |         console.warn("Upstash Redis get error, falling back to in-memory:", err);
0122 |       }
0123 |     }
0124 |
0125 |     return memoryPendingOffers.get(id);
0126 |   },
0127 |
0128 |   async remove(id: string): Promise<boolean> {
0129 |     const redis = getRedisClient();
0130 |     if (redis) {
0131 |       try {
0132 |         const existing = await redis.hget<IncomingOffer>(REDIS_PENDING_KEY, id);
0133 |         if (existing?.messageId) {
0134 |           await redis.sadd(REDIS_PROCESSED_KEY, existing.messageId);
0135 |         }
0136 |         const deletedCount = await redis.hdel(REDIS_PENDING_KEY, id);
0137 |         return deletedCount > 0;
0138 |       } catch (err) {
0139 |         console.warn("Upstash Redis remove error, falling back to in-memory:", err);
0140 |       }
0141 |     }
0142 |
0143 |     const existing = memoryPendingOffers.get(id);
0144 |     if (existing?.messageId) {
0145 |       memoryProcessedIds.add(existing.messageId);
0146 |       if (memoryProcessedIds.size > 500) {
0147 |         const oldest = memoryProcessedIds.values().next().value;
0148 |         if (oldest) memoryProcessedIds.delete(oldest);
0149 |       }
0150 |     }
```

```
0151 |     return memoryPendingOffers.delete(id);
0152 |   },
0153 |
0154 |   async clear(): Promise<void> {
0155 |     const redis = getRedisClient();
0156 |     if (redis) {
0157 |       try {
0158 |         const allOffers = (await redis.hvals<IncomingOffer>(REDIS_PENDING_KEY)) || [];
0159 |         const messageIds = allOffers.map((o) => o.messageId).filter(Boolean) as string[];
0160 |         if (messageIds.length > 0) {
0161 |           await redis.sadd(REDIS_PROCESSED_KEY, ...messageIds);
0162 |         }
0163 |         await redis.del(REDIS_PENDING_KEY);
0164 |         return;
0165 |       } catch (err) {
0166 |         console.warn("Upstash Redis clear error, falling back to in-memory:", err);
0167 |       }
0168 |     }
0169 |
0170 |     for (const offer of memoryPendingOffers.values()) {
0171 |       if (offer.messageId) memoryProcessedIds.add(offer.messageId);
0172 |     }
0173 |     memoryPendingOffers.clear();
0174 |   },
0175 |
0176 |   async reset(): Promise<void> {
0177 |     const redis = getRedisClient();
0178 |     if (redis) {
0179 |       try {
0180 |         await redis.del(REDIS_PENDING_KEY, REDIS_PROCESSED_KEY);
0181 |         return;
0182 |       } catch (err) {
0183 |         console.warn("Upstash Redis reset error, falling back to in-memory:", err);
0184 |       }
0185 |     }
0186 |
0187 |     memoryPendingOffers.clear();
0188 |     memoryProcessedIds.clear();
0189 |   },
0190 | };
```

Module 4: Primary Forensic Analysis Engine & Tiered LLMs

File Path:	src/app/api/analyze/route.ts
Module Role:	Integrates primary Groq LPU inference with automatic fallback to Google Gemini, regex-based UPI financial extortion extraction, and mathematical composite scoring.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	73 lines of code | TypeScript / Next.js

```
0001 | import { NextRequest, NextResponse } from "next/server";
0002 | import { analyzeOffer } from "@/services/geminiService";
0003 |
0004 | export async function POST(request: NextRequest) {
0005 |   try {
0006 |     const body = await request.json();
0007 |     const { text } = body;
0008 |
0009 |     if (!text || typeof text !== "string" || text.trim().length < 20) {
0010 |       return NextResponse.json(
0011 |         { error: "Please provide a valid offer text (at least 20 characters)." },
0012 |         { status: 400 }
0013 |       );
0014 |     }
0015 |
0016 |     // ■■ Gibberish / non-offer text detection ■■
0017 |     const trimmed = text.trim();
0018 |     const words = trimmed.split(/\s+/).filter((w: string) => w.length > 0);
0019 |     const wordCount = words.length;
0020 |
0021 |     // Must have at least 10 words to be a meaningful offer
0022 |     if (wordCount < 10) {
0023 |       return NextResponse.json(
0024 |         { error: "Please provide a valid offer letter or message. The text is too short to analyze." },
0025 |         { status: 400 }
0026 |       );
0027 |     }
0028 |
0029 |     // Check for real English words — common words found in any offer/email
0030 |     const commonWords = new Set([
0031 |       "the", "a", "an", "is", "are", "was", "were", "be", "been", "being",
0032 |       "have", "has", "had", "do", "does", "did", "will", "would", "could",
0033 |       "should", "may", "might", "shall", "can", "to", "of", "in", "for",
0034 |       "on", "with", "at", "by", "from", "this", "that", "it", "not", "but",
0035 |       "and", "or", "if", "so", "we", "you", "your", "our", "i", "my", "me",
0036 |       "he", "she", "they", "them", "his", "her", "its", "no", "yes",
0037 |       "dear", "hi", "hello", "please", "thank", "thanks", "regards",
0038 |       "company", "offer", "job", "internship", "role", "position", "salary",
0039 |       "payment", "pay", "work", "email", "contact", "team", "apply",
0040 |       "selected", "congratulations", "opportunity", "interview", "resume",
0041 |     ]);
0042 |     const recognizedWords = words.filter((w: string) =>
0043 |       commonWords.has(w.toLowerCase().replace(/[^a-z]/g, ""))
0044 |     );
0045 |     const recognizedRatio = recognizedWords.length / wordCount;
0046 |
0047 |     // If less than 15% of words are recognizable English, it's likely gibberish
0048 |     if (recognizedRatio < 0.15) {
0049 |       return NextResponse.json(
0050 |         { error: "Please provide a valid offer letter or message. The text appears to be random characters, not a real o
0051 |         { status: 400 }
0052 |       );
0053 |     }
0054 |
0055 |     // Check if text has at least some sentence-like structure (contains letters and spaces)
0056 |     const letterRatio = (trimmed.match(/[a-zA-Z]/g) || []).length / trimmed.length;
0057 |     if (letterRatio < 0.4) {
0058 |       return NextResponse.json(
0059 |         { error: "Please provide a valid offer letter or message. The text doesn't appear to contain readable content."
0060 |         { status: 400 }
0061 |       );
0062 |     }
0063 |
0064 |     const result = await analyzeOffer(text.trim());
0065 |     return NextResponse.json(result);
0066 |   } catch (error) {
0067 |     console.error("Analysis API Error:", error);
0068 |     return NextResponse.json(
0069 |       { error: "Failed to analyze the offer. Please try again." },
0070 |       { status: 500 }
0071 |     );
0072 |   }
0073 | }
```

Module 5: Asynchronous Ingestion & Event Listener API

File Path:	src/app/api/n8n/incoming/route.ts
Module Role:	Accepts webhook notifications from decentralized automation pipelines, computes payload cryptographic hashes, and queues offers for user review.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	125 lines of code | TypeScript / Next.js

```
0001 | import { NextRequest, NextResponse } from "next/server";
0002 | import { incomingStore } from "@/lib/incomingStore";
0003 |
0004 | export async function POST(request: NextRequest) {
0005 |   try {
0006 |     // Optional webhook secret verification
0007 |     const expectedSecret = process.env.N8N_WEBHOOK_SECRET;
0008 |     if (expectedSecret) {
0009 |       const providedSecret = request.headers.get("x-n8n-secret");
0010 |       if (providedSecret !== expectedSecret) {
0011 |         return NextResponse.json(
0012 |           { error: "Unauthorized: Invalid webhook secret" },
0013 |           { status: 401 }
0014 |         );
0015 |       }
0016 |     }
0017 |
0018 |     const body = await request.json();
0019 |     console.log("=== RAW N8N BODY ===", JSON.stringify(body, null, 2));
0020 |
0021 |     // Robust sender extraction
0022 |     let sender = body.sender || body.From || body.from || body.senderEmail;
0023 |     if (typeof sender === "object" && sender !== null) {
0024 |       sender = sender.text || sender.value?.[0]?.address || sender.value?.[0]?.name || sender.address;
0025 |     }
0026 |
0027 |     // Robust subject extraction
0028 |     let subject = body.subject || body.Subject || body.title;
0029 |
0030 |     const emailBody = body.body || body.textPlain || body.snippet || body.text || "";
0031 |     const snippet = body.snippet || (emailBody ? emailBody.slice(0, 150) : "No preview available");
0032 |
0033 |     // Fallback: If subject is missing, try extracting from the first line of body if it starts with "Subject:"
0034 |     if (!subject || subject === "No Subject") {
0035 |       const subjectMatch = emailBody.match(/Subject:\s*([^\r\n]+)/i);
0036 |       if (subjectMatch) {
0037 |         subject = subjectMatch[1].trim();
0038 |       } else {
0039 |         subject = "Internship / Job Offer";
0040 |       }
0041 |     }
0042 |
0043 |     // Fallback: If sender is missing, try extracting email from body
0044 |     if (!sender || sender === "Unknown Recruiter") {
0045 |       const emailMatch = emailBody.match(/([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})/);
0046 |       if (emailMatch) {
0047 |         sender = emailMatch[1];
0048 |       } else {
0049 |         sender = "Recruiter (via Gmail)";
0050 |       }
0051 |     }
0052 |
0053 |     const messageId = body.messageId || body.id || body.message_id;
0054 |
0055 |     if (!emailBody && !snippet) {
0056 |       return NextResponse.json(
0057 |         { error: "Invalid payload: email content missing" },
0058 |         { status: 400 }
0059 |       );
0060 |     }
0061 |
0062 |     const newOffer = await incomingStore.add({
0063 |       messageId: messageId ? String(messageId) : undefined,
0064 |       sender: String(sender),
0065 |       subject: String(subject),
0066 |       snippet: String(snippet),
0067 |       body: String(emailBody || snippet),
0068 |       receivedAt: body.receivedAt || new Date().toISOString(),
0069 |     });
0070 |
0071 |     if (!newOffer) {
0072 |       return NextResponse.json({
0073 |         success: true,
0074 |         message: "Offer already processed or queued (deduplicated)",
0075 |         duplicate: true,
```

```
0076 |       });
0077 |     }
0078 |
0079 |     return NextResponse.json({
0080 |       success: true,
0081 |       message: "Offer queued for user review & permission",
0082 |       offerId: newOffer.id,
0083 |     });
0084 |   } catch (error) {
0085 |     console.error("n8n Webhook Error:", error);
0086 |     return NextResponse.json(
0087 |       { error: "Failed to process incoming offer" },
0088 |       { status: 500 }
0089 |     );
0090 |   }
0091 | }
0092 |
0093 | // GET: Fetch all offers awaiting user permission
0094 | export async function GET() {
0095 |   const offers = await incomingStore.getAll();
0096 |   return NextResponse.json({ offers });
0097 | }
0098 |
0099 | // DELETE: Remove an offer once analyzed or dismissed (or clear all with id=all)
0100 | export async function DELETE(request: NextRequest) {
0101 |   try {
0102 |     const { searchParams } = new URL(request.url);
0103 |     const id = searchParams.get("id");
0104 |
0105 |     if (!id) {
0106 |       return NextResponse.json({ error: "Missing offer ID" }, { status: 400 });
0107 |     }
0108 |
0109 |     if (id === "reset") {
0110 |       await incomingStore.reset();
0111 |       return NextResponse.json({ success: true, cleared: "reset" });
0112 |     }
0113 |
0114 |     if (id === "all") {
0115 |       await incomingStore.clear();
0116 |       return NextResponse.json({ success: true, cleared: "all" });
0117 |     }
0118 |
0119 |     const removed = await incomingStore.remove(id);
0120 |     return NextResponse.json({ success: removed });
0121 |   } catch (error) {
0122 |     console.error("Delete Offer Error:", error);
0123 |     return NextResponse.json({ error: "Failed to remove offer" }, { status: 500 });
0124 |   }
0125 | }
```

Module 6: Interactive Forensic X-Ray Inspector

File Path:	src/components/views/XRayTextView.tsx
Module Role:	Interactive React component that renders the offer letter body with color-coded token highlights over identified predatory phrases, salary anomalies, and UPI demands.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	204 lines of code | TypeScript / Next.js

```
"use client";

import { useMemo } from "react";
import { HighlightedPhrase } from "@/lib/types";

interface XRayTextViewProps {
  originalText: string;
  highlights: HighlightedPhrase[];
}

const severityColors = {
  danger: "bg-red-500/20 text-red-950 dark:text-red-100 border-b-2 border-red-500 font-semibold px-1 py-0.5 rounded shad
  suspicious: "bg-amber-500/20 text-amber-950 dark:text-amber-100 border-b-2 border-amber-500 px-1 py-0.5 rounded",
  safe: "bg-emerald-500/20 text-emerald-950 dark:text-emerald-100 border-b-2 border-emerald-500 px-1 py-0.5 rounded",
};

function escapeRegex(str: string): string {
  return str.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
}

function findPhraseMatches(original: string, phrase: string): { start: number; end: number }[] {
  const matches: { start: number; end: number }[] = [];
  if (!phrase || !phrase.trim()) return matches;

  const lowerOriginal = original.toLowerCase();
  const lowerPhrase = phrase.toLowerCase().trim();

  // 1. Direct substring match
  let idx = lowerOriginal.indexOf(lowerPhrase);
  if (idx !== -1) {
    while (idx !== -1) {
      matches.push({ start: idx, end: idx + lowerPhrase.length });
      idx = lowerOriginal.indexOf(lowerPhrase, idx + 1);
    }
    return matches;
  }

  // 2. Whitespace-flexible regex (handles line breaks \n, \r\n, and multiple spaces)
  const words = lowerPhrase.split(/\s+/).filter(Boolean);
  if (words.length > 0) {
    const pattern = words.map(escapeRegex).join("[\\s\\r\\n]+");
    try {
      const reg = new RegExp(pattern, "gi");
      let m: RegExpExecArray | null;
      while ((m = reg.exec(original)) !== null) {
        matches.push({ start: m.index, end: m.index + m[0].length });
      }
      if (matches.length > 0) return matches;
    } catch {
      // fallback to next step
    }
  }

  // 3. Punctuation-tolerant fallback (strip trailing/leading punctuation)
  const cleanPhrase = lowerPhrase.replace(/^[.,:;!?'"“”‘’]+|[.,:;!?'"“”‘’]+$/g, "").trim();
  if (cleanPhrase !== lowerPhrase && cleanPhrase.length > 3) {
    const cleanWords = cleanPhrase.split(/\s+/).filter(Boolean);
    if (cleanWords.length > 0) {
      const pattern = cleanWords.map(escapeRegex).join("[\\s\\r\\n]+");
      try {
        const reg = new RegExp(pattern, "gi");
        let m: RegExpExecArray | null;
        while ((m = reg.exec(original)) !== null) {
          matches.push({ start: m.index, end: m.index + m[0].length });
        }
      } catch {
        // fallback
      }
    }
  }

  return matches;
}

export default function XRayTextView({ originalText, highlights }: XRayTextViewProps) {
```

```
0076 |   // Compute positions
0077 |   const { parts, counts } = useMemo(() => {
0078 |     const counts = { danger: 0, suspicious: 0, safe: 0 };
0079 |
0080 |     if (!highlights || highlights.length === 0 || !originalText) {
0081 |       return { parts: [<span key="raw">{originalText}</span>], counts };
0082 |     }
0083 |
0084 |     // Sort highlights: DANGER takes top priority, then suspicious, then safe, then longest first
0085 |     const severityWeight: Record<string, number> = { danger: 3, suspicious: 2, safe: 1 };
0086 |     const sortedHighlights = [...highlights].sort((a, b) => {
0087 |       const weightDiff = (severityWeight[b.severity] || 0) - (severityWeight[a.severity] || 0);
0088 |       if (weightDiff !== 0) return weightDiff;
0089 |       return b.text.length - a.text.length;
0090 |     });
0091 |
0092 |     const positions: {
0093 |       start: number;
0094 |       end: number;
0095 |       severity: "safe" | "suspicious" | "danger";
0096 |       text: string;
0097 |     }[] = [];
0098 |
0099 |     for (const highlight of sortedHighlights) {
0100 |       const matches = findPhraseMatches(originalText, highlight.text);
0101 |
0102 |       for (const match of matches) {
0103 |         // Check if this position overlaps with existing higher-priority positions
0104 |         const overlaps = positions.some(
0105 |           (p) => match.start < p.end && match.end > p.start
0106 |         );
0107 |
0108 |         if (!overlaps) {
0109 |           positions.push({
0110 |             start: match.start,
0111 |             end: match.end,
0112 |             severity: highlight.severity,
0113 |             text: originalText.slice(match.start, match.end),
0114 |           });
0115 |         }
0116 |       }
0117 |     }
0118 |
0119 |     // Count severities
0120 |     for (const p of positions) {
0121 |       if (p.severity in counts) {
0122 |         counts[p.severity]++;
0123 |       }
0124 |     }
0125 |
0126 |     // Sort positions by start index
0127 |     positions.sort((a, b) => a.start - b.start);
0128 |
0129 |     // Build parts
0130 |     const parts: React.ReactNode[] = [];
0131 |     let lastIndex = 0;
0132 |
0133 |     positions.forEach((pos, idx) => {
0134 |       // Add text before this highlight
0135 |       if (pos.start > lastIndex) {
0136 |         parts.push(
0137 |           <span key={`text-${idx}`}>
0138 |             {originalText.slice(lastIndex, pos.start)}
0139 |           </span>
0140 |         );
0141 |       }
0142 |
0143 |       // Add highlighted text
0144 |       parts.push(
0145 |         <mark
0146 |           key={`highlight-${idx}`}
0147 |           className={`${severityColors[pos.severity]} inline`}
0148 |           title={`${pos.severity.toUpperCase()} Indicator: "${pos.text.trim().replace(/\s+/g, " ").slice(0, 80)}"`}
0149 |         >
0150 |           {pos.text}
```

```
0151 |         </mark>
0152 |       );
0153 |
0154 |       lastIndex = pos.end;
0155 |     });
0156 |
0157 |     // Add remaining text
0158 |     if (lastIndex < originalText.length) {
0159 |       parts.push(
0160 |         <span key="text-end">{originalText.slice(lastIndex)}</span>
0161 |       );
0162 |     }
0163 |
0164 |     return { parts, counts };
0165 |   }, [originalText, highlights]);
0166 |
0167 |   return (
0168 |     <div className="rounded-xl border border-border bg-card p-6 shadow-sm space-y-4">
0169 |       {/* Header and Legend */}
0170 |       <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-3 pb-3 border-b border-border/60">
0171 |         <div className="flex items-center gap-2.5">
0172 |           <h3 className="text-lg font-bold text-foreground tracking-tight flex items-center gap-2">
0173 |             <span>■</span>
0174 |             <span>X-Ray Forensic Analysis</span>
0175 |           </h3>
0176 |           <span className="text-xs text-muted-foreground font-normal">
0177 |             (Interactive Word-Level Indicators)
0178 |           </span>
0179 |         </div>
0180 |
0181 |         {/* Legend with active counts */}
0182 |         <div className="flex items-center gap-3 text-xs font-medium flex-wrap">
0183 |           <span className="flex items-center gap-1.5 px-2 py-1 rounded-md bg-red-500/10 text-red-600 dark:text-red-400 b
0184 |             <span className="w-2.5 h-2.5 rounded-full bg-red-500 animate-pulse" />
0185 |             <span>Danger ({counts.danger})</span>
0186 |           </span>
0187 |           <span className="flex items-center gap-1.5 px-2 py-1 rounded-md bg-amber-500/10 text-amber-600 dark:text-amber
0188 |             <span className="w-2.5 h-2.5 rounded-full bg-amber-500" />
0189 |             <span>Suspicious ({counts.suspicious})</span>
0190 |           </span>
0191 |           <span className="flex items-center gap-1.5 px-2 py-1 rounded-md bg-emerald-500/10 text-emerald-600 dark:text-e
0192 |             <span className="w-2.5 h-2.5 rounded-full bg-emerald-500" />
0193 |             <span>Safe ({counts.safe})</span>
0194 |           </span>
0195 |         </div>
0196 |       </div>
0197 |
0198 |       {/* Highlighted text container */}
0199 |       <div className="text-sm leading-relaxed whitespace-pre-wrap font-sans text-foreground/90 p-3 bg-muted/20 rounded-l
0200 |         {parts}
0201 |       </div>
0202 |     </div>
0203 |   );
0204 | }
```

Module 7: Client-Side Forensic State Management

File Path:	src/store/offerStore.ts
Module Role:	Zustand state store managing active offer triage, heuristic breakdown metrics, placement cell reporting drafts, and mock forensic test cases.
Authors:	Soham Jadhav, Amey Bauchkar (University of Mumbai)
Metrics:	47 lines of code | TypeScript / Next.js

```
0001 | import { create } from "zustand";
0002 | import { AnalysisResult } from "@/lib/types";
0003 |
0004 | interface OfferStore {
0005 |   inputText: string;
0006 |   isAnalyzing: boolean;
0007 |   result: AnalysisResult | null;
0008 |   error: string | null;
0009 |   setInputText: (text: string) => void;
0010 |   analyzeOffer: (text: string) => Promise<void>;
0011 |   reset: () => void;
0012 | }
0013 |
0014 | export const useOfferStore = create<OfferStore>((set) => ({
0015 |   inputText: "",
0016 |   isAnalyzing: false,
0017 |   result: null,
0018 |   error: null,
0019 |
0020 |   setInputText: (text: string) => set({ inputText: text }),
0021 |
0022 |   analyzeOffer: async (text: string) => {
0023 |     set({ isAnalyzing: true, result: null, error: null });
0024 |     try {
0025 |       const response = await fetch("/api/analyze", {
0026 |         method: "POST",
0027 |         headers: { "Content-Type": "application/json" },
0028 |         body: JSON.stringify({ text }),
0029 |       });
0030 |
0031 |       if (!response.ok) {
0032 |         const errorData = await response.json();
0033 |         throw new Error(errorData.error || "Analysis failed");
0034 |       }
0035 |
0036 |       const result: AnalysisResult = await response.json();
0037 |       set({ result, isAnalyzing: false });
0038 |     } catch (error) {
0039 |       set({
0040 |         error: error instanceof Error ? error.message : "Something went wrong",
0041 |         isAnalyzing: false,
0042 |       });
0043 |     }
0044 |   },
0045 |
0046 |   reset: () => set({ inputText: "", result: null, error: null, isAnalyzing: false }),
0047 | }));
```